ANÁLISIS DE DATOS – INTEGRADOR

Misión 1



TEMA 1: INTRODUCCIÓN A BASES DE DATOS Y SQL



Misión 1

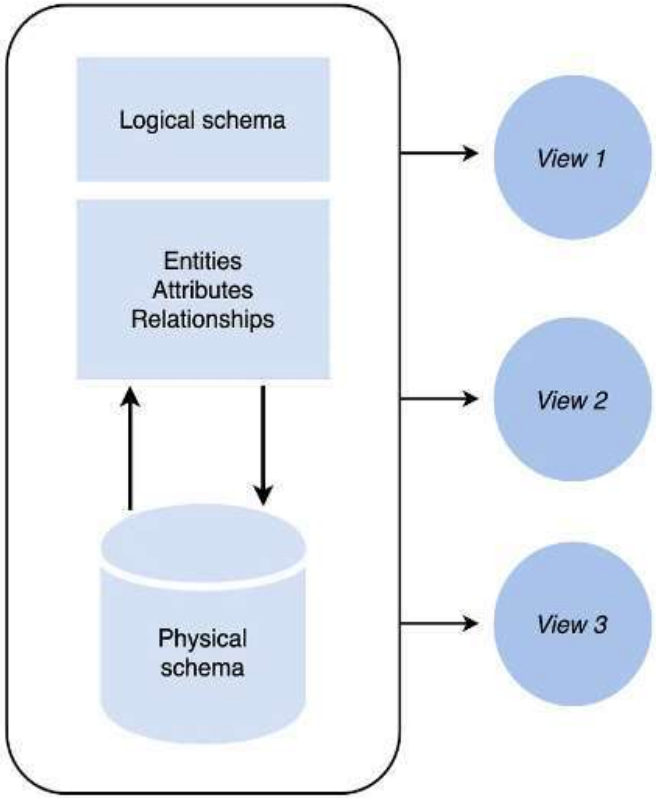
¿QUE ES SQL?

- SQL es un lenguaje estándar para acceder y manipular bases de datos.
- SQL son las siglas de Structured Query Language
- SQL le permite acceder y manipular bases de datos
- SQL se convirtió en un estándar del American National Standards Institute (ANSI) en 1986 y de la Organización Internacional de Normalización (ISO) en 1987.

¿QUE PUEDO HACER CON SQL?

- SQL puede ejecutar consultas en una base de datos
- SQL puede recuperar datos de una base de datos
- SQL puede insertar registros en una base de datos
- SQL puede actualizar registros en una base de datos
- SQL puede eliminar registros de una base de datos
- SQL puede crear nuevas bases de datos
- SQL puede crear nuevas tablas en una base de datos
- SQL puede crear procedimientos almacenados en una base de datos
- SQL puede crear vistas en una base de datos

CONCEPTOS BÁSICOS DE LAS BASES DE DATOS



Estructura

- Las bases de datos se organizan en tablas con filas (registros) y columnas (atributos).

Tipos de datos

- Cada columna almacena un tipo de dato específico como texto, números, fechas o valores booleanos.

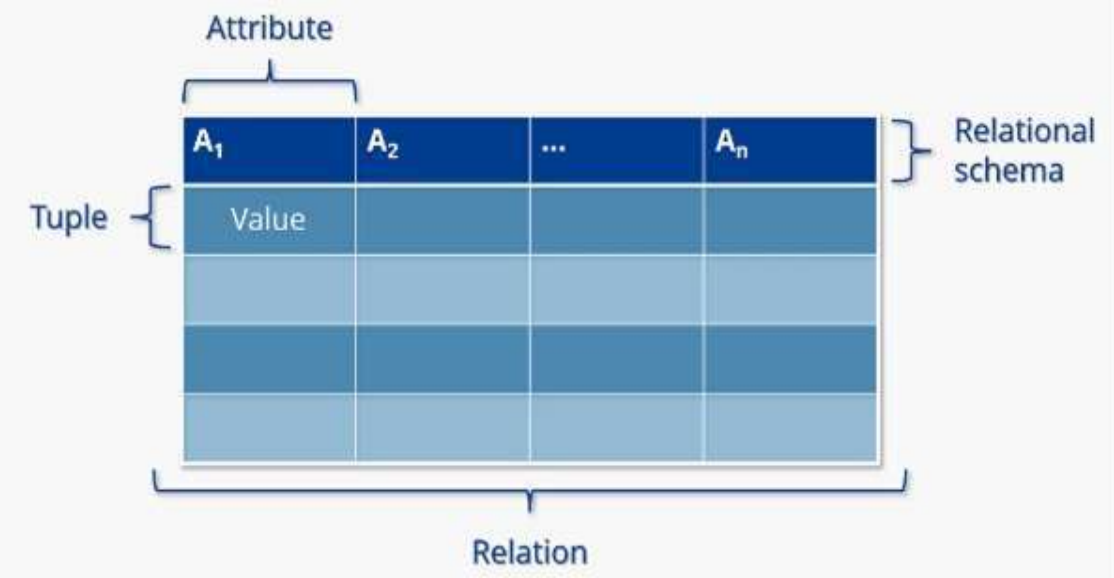
Claves

- Las claves identifican de forma única cada registro en una tabla, asegurando la integridad de los datos.

Relación

- Las tablas se relacionan entre sí mediante claves foráneas, creando un modelo de datos complejo.

MODELOS DE DATOS



Modelo Relacional

- Se basa en tablas relacionadas, ideal para organizar información compleja.

Modelo NoSQL

- Ofrece flexibilidad para datos estructurados y no estructurados, adecuado para aplicaciones web.

Modelo de Objetos

- Utiliza objetos como entidades principales, ideal para desarrollo orientado a objetos.

Modelo de Gráficos

- Representa datos como nodos y relaciones, ideal para redes sociales y análisis de datos.

Instalación y Configuración de MySQL



1

Descarga

Descargue la versión de MySQL adecuada para su sistema operativo desde la página web oficial.

2

Instalación

Siga las instrucciones del instalador para instalar MySQL en su computadora.

3

Configuración

Ajuste los parámetros de configuración del servidor MySQL para optimizar su rendimiento.

4

Acceso

Conecte a la base de datos de MySQL usando un cliente de línea de comandos o una herramienta gráfica.

CREACIÓN DE BASES DE DATOS

Conectar al Servidor

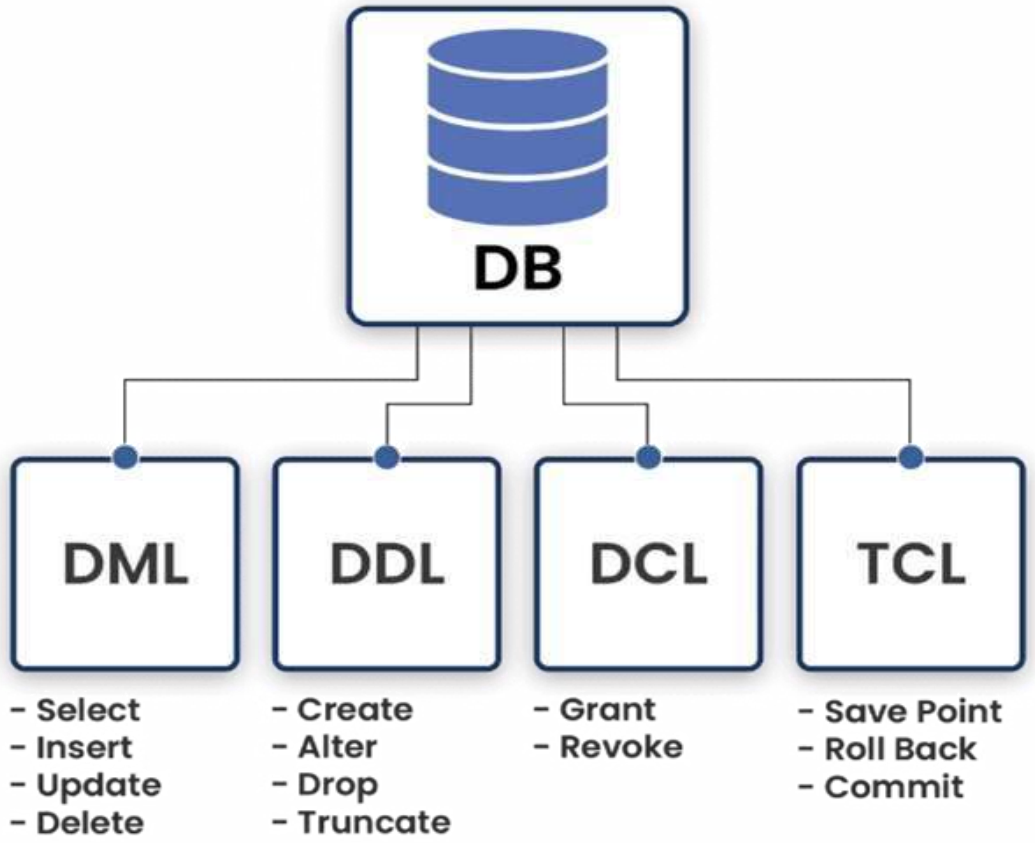
- Establezca una conexión al servidor de bases de datos (MySQL o PostgreSQL).

Crear la Base de Datos

- Utilice la instrucción CREATE DATABASE para crear una nueva base de datos.

Seleccionar la Base de Datos

- Utilice la instrucción USE para seleccionar la base de datos con la que desea trabajar.



EJEMPLO DE BASE DE DATOS.

"**Ciencia, tecnología e innovación:** Un sistema que almacene datos sobre nuevas patentes en biotecnología, su aplicación en la industria y la inversión en I+D por parte de diferentes sectores."

TIPOS DE DATOS SQL

Tipos de datos de cadena	
Datatype	**Descripción**
CHAR(tamaño)	Puede contener letras, números y caracteres especiales, puede ser de 0 a 255. El valor predeterminado es 1
VARCHAR(tamaño)	Puede contener letras, números y caracteres especiales, puede ser de 0 a 65535
Tipos de datos numéricos	
Datatype	**Descripción**
INTEGER(tamaño)	Número entero, el parámetro de tamaño especifica el ancho máximo de visualización que es 255
DOUBLE(tamaño, decimas)	Número de coma flotante, El número total de dígitos se especifica en tamaño. El número de dígitos después del punto decimal se especifica en el parámetro d
Tipos de datos de fecha y hora	
Datatype	**Descripción**
DATE	Formato: YYYY-MM-DD
DATETIME(fsp)	Formato: AAAA-MM-DD hh: mm: ss.
TIME(fsp)	Formato: hh:mm:ss

LA INSTRUCCIÓN SQL CREATE TABLE

La sentencia CREATE TABLE se utiliza para crear una nueva tabla en una base de datos.

CREATE TABLE table_name (column1 datatype, ..., columnN datatype);

CREATE TABLE Personas (PersonaID int, Nombre varchar(55), Direccion varchar(55), Ciudad varchar(55));

CLAVES PRIMARIAS Y FORÁNEAS

Claves Primarias (PK)

Identifican de forma única cada fila en una tabla.

Claves Foráneas (FK)

Hacen referencia a las claves primarias de otras tablas, estableciendo relaciones entre ellas.

Integridad Referencial

Garantiza que las claves foráneas siempre hagan referencia a claves primarias válidas.

RESTRICCIÓN DE PRIMARY KEY

La restricción PRIMARY KEY identifica de forma única cada registro en una tabla. Las claves primarias deben contener valores ÚNICOS y no pueden contener valores NULL. Una tabla solo puede tener UNA clave primaria.

El siguiente SQL crea una PRIMARY KEY en la columna "ID" cuando se crea la tabla "Personas"

```
CREATE TABLE Personas (ID int (3), Nombre varchar(55), Edad int, PRIMARY KEY (ID));
```

Para crear una restricción PRIMARY KEY en la columna "ID" cuando la tabla ya está creada, use el siguiente SQL

```
ALTER TABLE Personas ADD PRIMARY KEY (ID);
```

Para eliminar una restricción PRIMARY KEY, use el siguiente SQL

```
ALTER TABLE Personas DROP PRIMARY KEY;
```

LA INSTRUCCIÓN FOREIGN KEY

La FOREIGN KEY restricción se utiliza para evitar acciones que destruyan enlaces entre tablas.

A FOREIGN KEY es un campo (o colección de campos) en una tabla, que se refiere a PRIMARY KEY en otra tabla.

La siguiente instrucción crea una FOREIGN KEY desde el CREATE TABLE

CREATE TABLE Ordenes (ID int NOT NULL, OrdenNum int NOT NULL, PersonaID int, PRIMARY KEY (OrderID), FOREIGN KEY (PersonID) REFERENCES Persona(PersonaID));

La siguiente instrucción crea una FOREIGN KEY después de que la tabla a sido creada

ALTER TABLE Ordenes ADD FOREIGN KEY (PersonaID) REFERENCES Persona(PersonaID);

Para eliminar una FOREIGN KEY

ALTER TABLE Ordenes DROP FOREIGN KEY FK_PersonaOrdenes;

SENTENCIA SQL DROP TABLE

La instrucción DROP TABLE se utiliza para eliminar una tabla existente en una base de datos.

DROP TABLE table_name;

La instrucción TRUNCATE TABLE se usa para eliminar los datos dentro de una tabla, pero no la tabla en sí.

TRUNCATE TABLE table_name;

SENTENCIA ALTER TABLE

La instrucción ALTER TABLE se usa para agregar, eliminar o modificar columnas en una tabla existente. La instrucción ALTER TABLE también se usa para agregar y eliminar varias restricciones en una tabla existente.

Para agregar una columna en una tabla, use la siguiente sintaxis:

ALTER TABLE table_name ADD column_name datatype;

Para eliminar una columna de una tabla, utilice la siguiente sintaxis:

ALTER TABLE table_name DROP COLUMN column_name;

Para cambiar el tipo de datos de una columna en una tabla, use la siguiente sintaxis:

ALTER TABLE table_name MODIFY COLUMN column_name datatype;

RESTRICCIÓN NOT NULL

De forma predeterminada, una columna puede contener valores NULL. La restricción NOT NULL obliga a una columna a NO aceptar valores NULL. Esto obliga a un campo a contener siempre un valor, lo que significa que no puede insertar un nuevo registro o actualizar un registro sin agregar un valor a este campo.

El siguiente SQL garantiza que las columnas "ID", y "Nombre" NO aceptarán valores NULL cuando se cree la tabla "Personas"

```
CREATE TABLE Personas (ID int NOT NULL, Nombre varchar(55) NOT NULL, Edad int);
```

Para crear una restricción NOT NULL en la columna "Edad" cuando la tabla "Personas" ya está creada, use el siguiente SQL

```
ALTER TABLE Personas MODIFY Edad int NOT NULL;
```

RESTRICCIÓN UNIQUE

La restricción UNIQUE asegura que todos los valores de una columna sean diferentes. Tanto las restricciones UNIQUE como PRIMARY KEY proporcionan una garantía de unicidad para una columna o un conjunto de columnas. Una restricción PRIMARY KEY tiene automáticamente una restricción UNIQUE. Sin embargo, puede tener muchas restricciones UNIQUE por tabla, pero solo una restricción PRIMARY KEY por tabla.

El siguiente SQL crea una restricción UNIQUE en la columna "ID" cuando se crea la tabla "Persona".

```
CREATE TABLE Persona (ID int NOT NULL, Nombre varchar(55), Edad int, UNIQUE (ID));
```

Para nombrar una restricción UNIQUE y definir una restricción UNIQUE en varias columnas, use la siguiente sintaxis SQL

```
CREATE TABLE Persons (ID int NOT NULL, Nombre varchar(55), Edad int, CONSTRAINT UC_Person UNIQUE (ID,Nombre));
```

RESTRICCIÓN UNIQUE

Para crear una restricción UNIQUE en la columna "ID" cuando la tabla ya está creada, use el siguiente SQL

```
ALTER TABLE Personas ADD UNIQUE (ID);
```

Para nombrar una restricción UNIQUE y definir una restricción UNIQUE en varias columnas, use la siguiente sintaxis SQL

```
ALTER TABLE Personas ADD CONSTRAINT UC_Persona UNIQUE (ID,LastName);
```

Para eliminar una restricción UNIQUE, use el siguiente SQL

```
ALTER TABLE Personas DROP INDEX UC_Persona;
```

ALGUNAS DE LAS SENTENCIAS DDL MÁS COMUNES

CREATE

- Utilizada para crear una nueva tabla, vista, índice, base de datos, etc. Por ejemplo, CREATE TABLE crea una nueva tabla.

ALTER

- Permite modificar la estructura de un objeto de base de datos existente, como añadir, borrar o modificar columnas en una tabla existente o modificar las propiedades de un objeto.

DROP

- Se usa para eliminar objetos de la base de datos, como tablas, vistas, índices, etc. Elimina completamente el objeto y los datos asociados.

TRUNCATE

- Elimina todos los registros de una tabla sin eliminar la tabla en sí. Es una manera rápida de limpiar los datos de una tabla.

COMMENT

- Permite añadir comentarios descriptivos a la definición de un objeto de la base de datos.

RENAME

- Cambia el nombre de un objeto de base de datos.



Creación de Tablas

1

Definir la Estructura

Especifica el nombre de la tabla y las columnas que contendrá.

2

Establecer Tipos de Datos

Asigna un tipo de dato a cada columna (texto, números, fechas, etc.).

3

Definir Claves

Especifica la clave principal y las claves foráneas para relacionar las tablas.

ALGUNAS DE LAS SENTENCIAS DML MÁS COMUNES

SELECT

- Se utiliza para consultar y recuperar datos de una base de datos. Puede ser tan simple como seleccionar todas las columnas de una tabla o tan compleja como incluir varias tablas, condiciones, agrupaciones, y ordenaciones.
- **SELECT** columna1, columna2 **FROM** tabla **WHERE** condicion;

INSERT

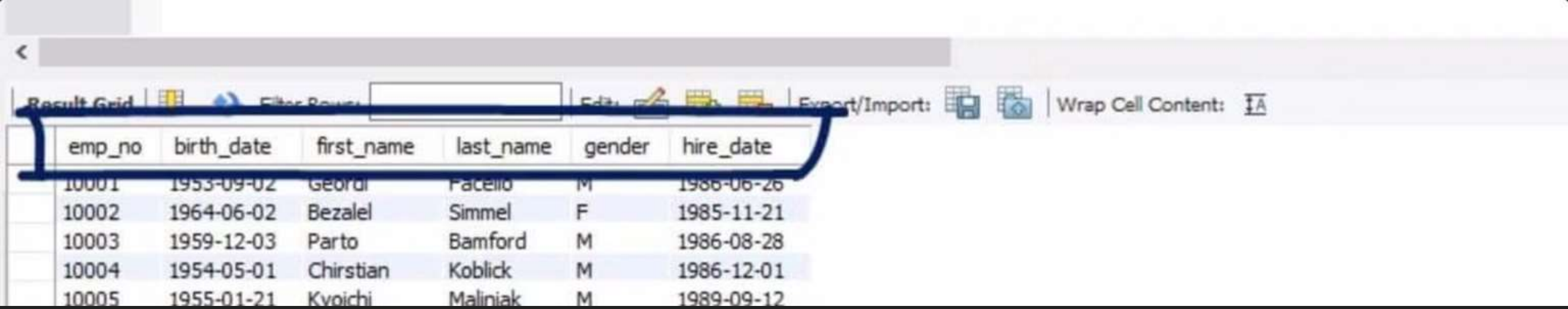
- Permite insertar nuevos registros en una tabla. Puedes especificar valores específicos para cada columna en la nueva fila.
- **INSERT INTO** tabla (columna1, columna2) **VALUES** (valor1, valor2);

UPDATE

- Modifica los datos existentes en una tabla. Puedes actualizar el valor de una o varias columnas en todas las filas que cumplan con una condición especificada.
- **UPDATE** tabla **SET** columna1 = valor1 **WHERE** condicion;

DELETE

- Elimina registros de una tabla basándose en una condición. Si no se especifica condición, todos los registros de la tabla pueden ser eliminados.
- **DELETE FROM** tabla **WHERE** condicion;



Comando INSERT

1

2

3

Inserta una nueva fila en una tabla.

Especifica el nombre de la tabla y los valores para cada columna.

Sintaxis:

INSERT INTO tabla (columna1, columna2, ...) VALUES (valor1, valor2, ...);

Ejemplo:

INSERT INTO Clientes (Nombre, Apellido, Ciudad) VALUES ('Ana', 'Lopez', 'Madrid');

Made with Gamma

LA INSTRUCCIÓN INSERT INTO

La instrucción INSERT INTO se utiliza para insertar nuevos registros en una tabla.

Sintanxis

INSERT INTO table_name (column1, columnN) VALUES (value1, valueN);

Ejemplo

INSERT INTO Estudiante (Id, Nombre) VALUES (‘101’, ‘Carlos’);

Comando UPDATE

1

Modifica valores existentes en una tabla.

Especifica el nombre de la tabla, las columnas a actualizar y las nuevas condiciones.

2

Sintaxis:

UPDATE tabla SET columna1 = valor1, columna2 = valor2, ... WHERE condición;

3

Ejemplo:

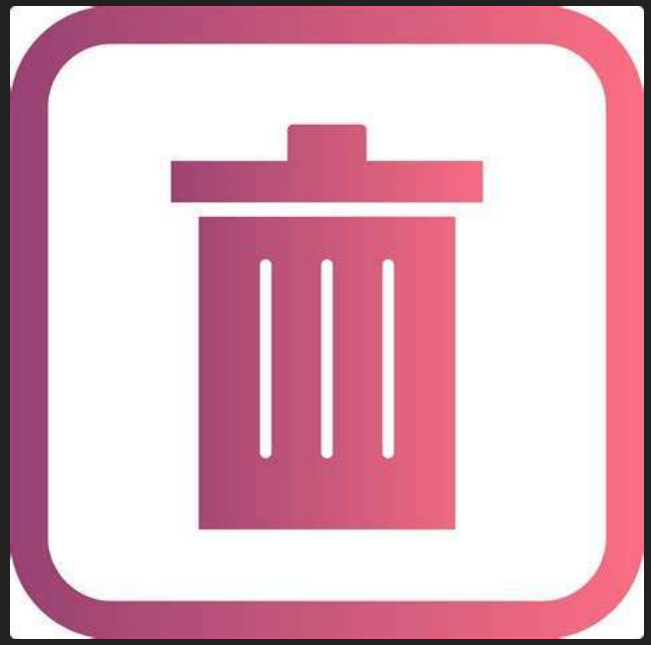
UPDATE Clientes SET Ciudad = 'Barcelona' WHERE Nombre = 'Ana';



Comando DELETE

Elimina filas de una tabla.

Especifica el nombre de la tabla y la condición para eliminar filas.



Sintaxis:

DELETE FROM tabla WHERE condición;

Precaución:

Elimina filas de forma permanente.

TEMA 2: CONSULTAS SQL BÁSICAS

Misión 1



Consultas Básicas de SQL

1 Sentencia SELECT

La sentencia **SELECT** se usa para recuperar datos de una o varias tablas.

2 Claúsula WHERE

La cláusula **WHERE** filtra los datos según condiciones especificadas.

3 Claúsula ORDER BY

La cláusula **ORDER BY** ordena los resultados según un criterio especificado.

4 Claúsula LIMIT

La cláusula **LIMIT** limita el número de filas devueltas.

LA INSTRUCCIÓN SELECT

La instrucción SELECT se utiliza para seleccionar datos de una base de datos.

Los datos devueltos se almacenan en una tabla de resultados, denominada conjunto de resultados.

Sintaxis

SELECT column1, column2 FROM table_name;

Ejemplo

SELECT * FROM Pacientes; (* selecciona todas las columnas de la tabla Pacientes)

SELECT Nombre, Telefono FROM Pacientes; (Nombre, Telefono columnas seleccionadas)

ACTIVIDAD BÁSICA SELECT

- Obtener los nombres y especialidades de los doctores
- Ver los nombres de los servicios y su precio
- Mostrar todas las citas registradas
- Mostrar los nombres y fechas de nacimiento de los doctores.
- Listar solo los nombres de los servicios y su duración estimada.
- Obtener todas las columnas de la tabla Servicios

CLÁUSULA WHERE

La WHERE cláusula se utiliza para filtrar registros. Se utiliza para extraer solo aquellos registros que cumplen una condición específica.

La WHERE cláusula no sólo se utiliza en SELECT , sino que también se utiliza en UPDATE, DELETE.

Sintaxis

```
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

EJEMPLO DE LA CLÁUSULA WHERE

En el siguiente ejemplo solo se mostrarán los nombres de las personas que sean de sexo femenino

```
SELECT * FROM Pacientes WHERE documento_identidad = '12345678';
SELECT nombre FROM Doctores WHERE especialidad = 'Cardiología';
```

La letra “s” es un alisas que se le asigna a la tabla servicios

	then condition becomes true.	
<	If the value of the left operand is less than the value of right operand.	(a < b) is true

Operadores de Comparación

Operador	Descripción	Ejemplo
=	Igual	Edad = 30
<>	Diferente	Nombre <> 'Juan'
>	Mayor que	Salario > 50000
<	Menor que	Fecha < '2023-12-31'
>=	Mayor o igual que	Puntos >= 100
<=	Menor o igual que	Talla <= 40

ACTIVIDAD BÁSICA WHERE

- Citas que están confirmadas
- Ver las citas que están pendientes
- Nombre de los servicios y su precio lo cuales tengan un precio mayor a 70
- Mostrar el nombre y especialidad del doctor que su nombre sea distinto a “Dr. Alejandro Rivas”

ORDER BY DE SQL

La ORDER BY palabra clave se utiliza para ordenar el conjunto de resultados en orden ascendente o descendente.

La ORDER BY palabra clave ordena los registros en orden ascendente de forma predeterminada. Para ordenar los registros en orden descendente, use la DESC palabra clave.

Sintaxis

```
SELECT column1, columnN
FROM table_name
ORDER BY column1 ASC|DESC;
```

La instrucción "|" representa que se debe escoger ASC o DESC, pero no ambas.

EJEMPLO ORDER BY

La siguiente instrucción SQL selecciona el nombre de los pacientes de la tabla “pacientes", ordenados por la columna "Nombre"

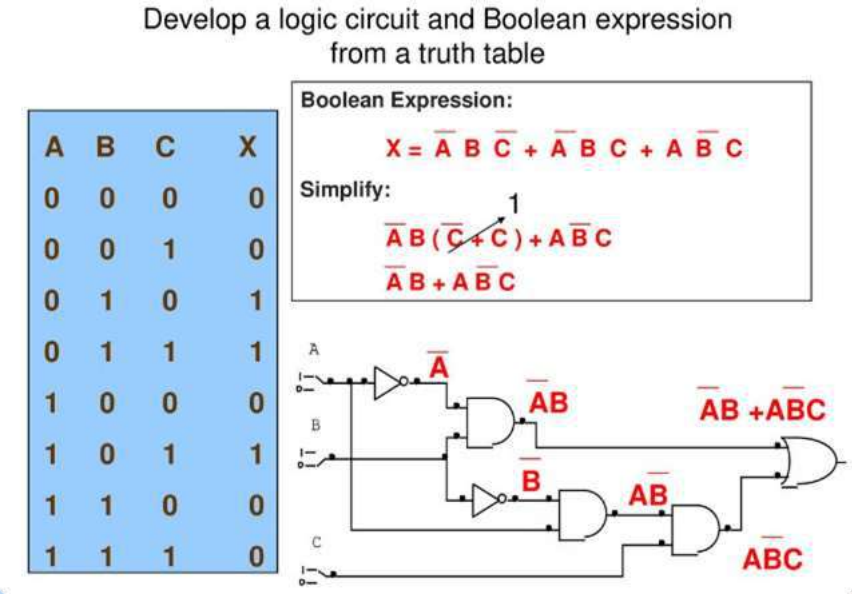
```
SELECT nombre FROM pacientes ORDER BY nombre;
```

La siguiente instrucción SQL selecciona todos los pacientes de la tabla “pacientes", en orden DESCENDIENTE por la columna "Nombre"

```
SELECT * FROM pacientes p
ORDER BY p.nombre DESC;
```

ACTIVIDAD BÁSICA ORDER BY

- Listar los nombres de los doctores y su fecha de nacimiento según su fecha de nacimiento (de los más jóvenes a los más viejos)
- Mostrar los nombre de los servicios y su precio del más barato al más caro
- Listar las citas por fecha (las más cercanas primero)
- Ver los doctores de "Cardiología" ordenados por edad (del mayor al menor)
- Ver las citas pendientes, ordenadas por la fecha más próxima.



Operadores Lógicos

AND

Devuelve **TRUE** si ambas condiciones son **TRUE**.

OR

Devuelve **TRUE** si al menos una condición es **TRUE**.

NOT

Invierte el valor lógico de una condición.

LOS OPERADORES AND, OR Y NOT

La WHERE cláusula se puede combinar con AND, OR y NOT operadores.

Los operadores AND y OR se utilizan para filtrar registros en función de más de una condición:

El AND operador muestra un registro si todas las condiciones separadas por AND son VERDADERAS.

El OR operador muestra un registro si alguna de las condiciones separadas por OR es VERDADERA.

El NOT operador muestra un registro si la(s) condición(es) NO es VERDADERA.

SINTAXIS AND, OR, NOT

Sintaxis AND

```
SELECT column1, columnN
FROM table_name
WHERE condition1 AND condition2 AND conditionN;
```

Sintaxis OR

```
SELECT column1, columnN
FROM table_name
WHERE condition1 OR condition2 OR conditionN;
```

Sintaxis NOT

```
SELECT column1, columnN
FROM table_name
WHERE NOT condition;
```

EJEMPLO OPERADORES AND, OR, NOT

Servicios que cuestan más de 70 Y duran más de 40 minutos

```
SELECT nombre_servicio, precio, duracion_estimada FROM Servicios
WHERE precio > 70 AND duracion_estimada > 40;
```

Citas que están pendientes O canceladas

```
SELECT * FROM Citas
WHERE estado = 'Pendiente' OR estado = 'Cancelada';
```

OPERADOR SQL BETWEEN

El BETWEEN operador selecciona valores dentro de un rango determinado. Los valores pueden ser números, texto o fechas.

Sintaxis

```
SELECT column1, columnN
FROM table_name
WHERE column_name BETWEEN value1 AND value2;
```

Ejemplo

- Pacientes nacidos entre 1985 y 1995

```
SELECT nombre, fecha_nacimiento FROM Pacientes
WHERE fecha_nacimiento BETWEEN '1985-01-01' AND '1995-12-31';
```

- Servicios con precios entre 50 y 90

```
SELECT nombre_servicio, precio FROM Servicios
WHERE precio BETWEEN 50 AND 90;
```

ACTIVIDAD BÁSICA OPERADORES (AND, OR, NOT) BETWEEN

- Citas confirmadas entre el 20 y 26 de marzo, ordenadas por fecha
- Obtener los servicios cuyo precio está entre 40 y 100.
- Pacientes nacidos entre 1980 y 2000 que tienen teléfono registrado

OPERADOR SQL LIKE

El LIKE operador se usa en una WHERE cláusula para buscar un patrón específico en una columna.

Hay dos comodines que se utilizan a menudo junto con el LIKE operador:

El signo de porcentaje (%) representa cero, uno o varios caracteres

El signo de subrayado (_) representa un solo carácter

```
SELECT column1, columnN, ...
FROM table_name
WHERE condición LIKE pattern;
```

EJEMPLOS OPERADOR LIKE

La siguiente instrucción SQL selecciona todos los pacientes con que comienza con "a":

```
SELECT nombre FROM Pacientes WHERE nombre LIKE 'A%';
```

Buscar servicios que contienen la palabra "Consulta"

```
SELECT nombre_servicio FROM Servicios WHERE nombre_servicio LIKE '%Consulta%’;
```

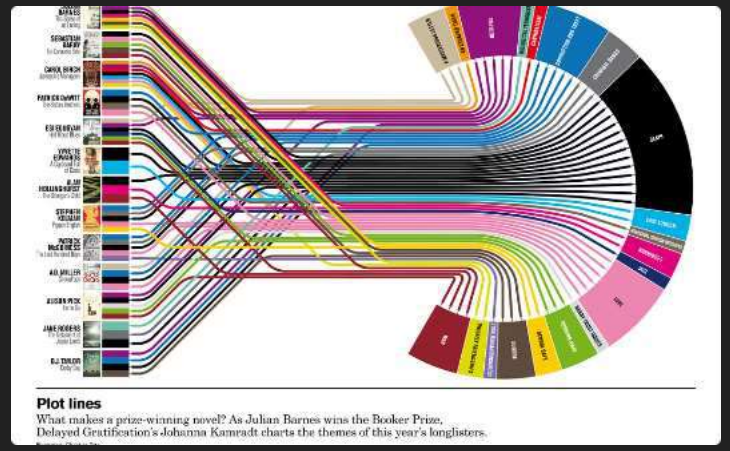
La siguiente instrucción SQL selecciona a todos los que NO comienza con "a":

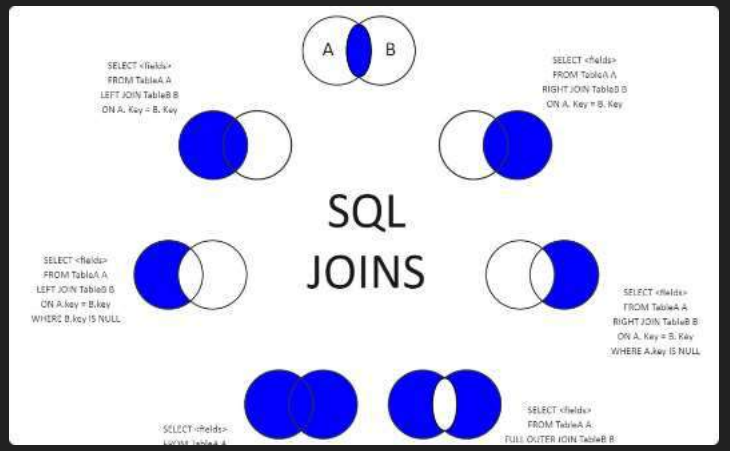
```
SELECT * FROM pacientes p WHERE p.nombre NOT LIKE 'a%';
```

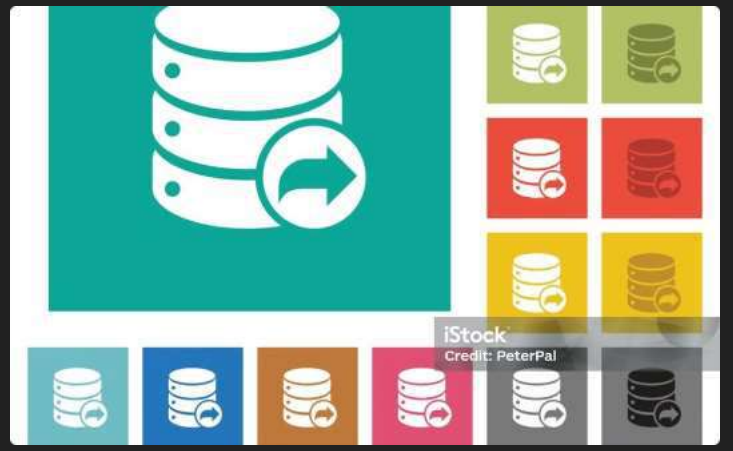
ACTIVIDAD BÁSICA LIKE

- Que hace la siguiente instrucción SQL:
  - SELECT nombre FROM Pacientes WHERE nombre LIKE '_____’;
- Buscar doctores cuyo nombre empieza con "Dr." Y ordenar por especialidad
- Mostrar las citas de pacientes cuyo nombre empieza con "J" o "P"

Funcionalidades Avanzadas







Funciones Agregadas

Calcula valores agregados como **SUM(), AVG(), MAX(), MIN() y COUNT().**

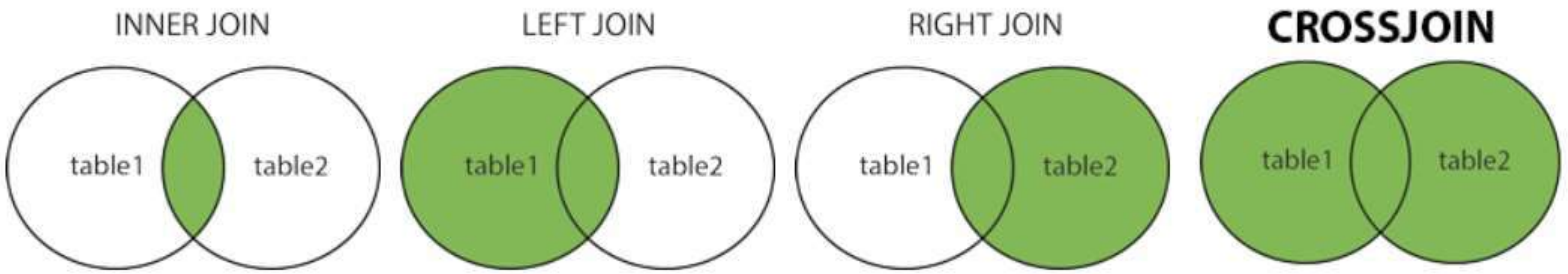
Uniones (JOIN)

Combinan datos de dos o más tablas basadas en una columna común.

Transacciones

Garantizan la consistencia de los datos, asegurando que las operaciones se completen en su totalidad.

TIPO DE JOIN ADMITIDOS



INNER JOIN

La palabra reservada ***INNER JOIN*** selecciona registros que tienen valores coincidentes en ambas tablas.

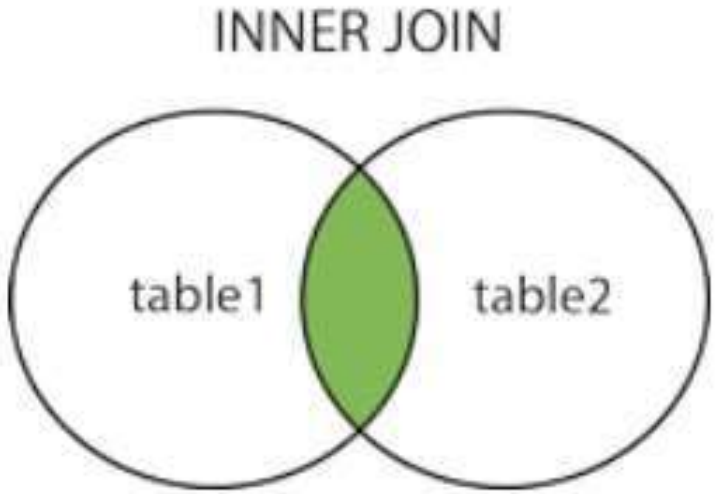
Sintaxis

SELECT columna_nombre(s)

FROM tabla1

INNER JOIN tabla2

ON tabla1.columna_nombre = tabla2.columna_nombre;



EJEMPLO INNER JOIN → COINCIDENCIAS EN AMBAS TABLAS

```
SELECT Citas.id, Pacientes.nombre AS paciente,
Doctores.nombre AS doctor, Citas.fecha, Citas.hora

FROM Citas

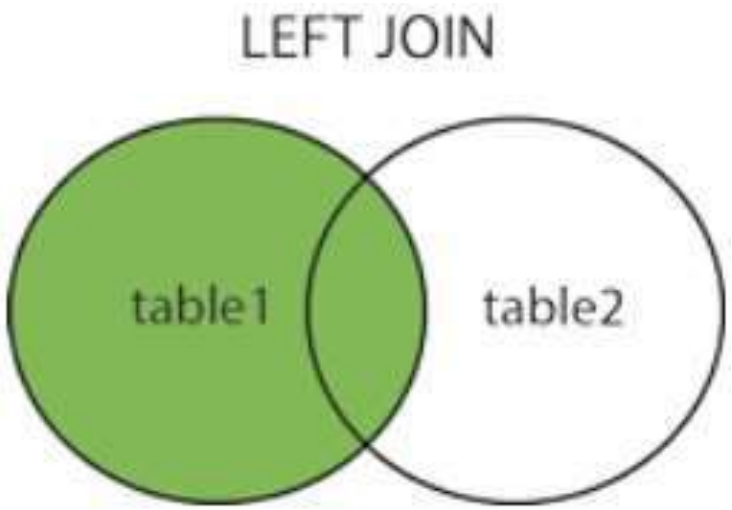
INNER JOIN Pacientes ON Citas.paciente_id =
Pacientes.id

INNER JOIN Doctores ON Citas.doctor_id =
Doctores.id;
```

- INNER JOIN devuelve solo las citas que tienen paciente y doctor existentes.
- Si hay citas sin paciente o doctor, no se mostrarán.

Cuándo usarlo: Cuando queremos solo datos relacionados entre tablas.

LEFT JOIN



La palabra reservada ***LEFT JOIN*** devuelve todos los registros de la tabla izquierda (tabla1) y los registros coincidentes (si los hay) de la tabla derecha (tabla2).

Sintaxis

SELECT columna_nombre(s)

FROM tabla1

LEFT JOIN tabla2

ON tabla1.columna_nombre = tabla2.columna_nombre;

EJEMPLO LEFT JOIN → TODOS LOS DE LA IZQUIERDA, COINCIDENCIAS EN LA DERECHA

```
SELECT Pacientes.nombre, Citas.fecha, Citas.hora
FROM Pacientes
LEFT JOIN Citas ON Pacientes.id = Citas.paciente_id;
```

- Muestra todos los pacientes.
- Si un paciente no tiene cita, los valores de Citas aparecerán como NULL.

Cuándo usarlo: Cuando queremos ver todos los pacientes, incluso si no tienen citas.

RIGHT JOIN

La palabra reservada ***RIGHT JOIN*** devuelve todos los registros de la tabla de la derecha (tabla2) y los registros coincidentes (si los hay) de la tabla de la izquierda (tabla1).

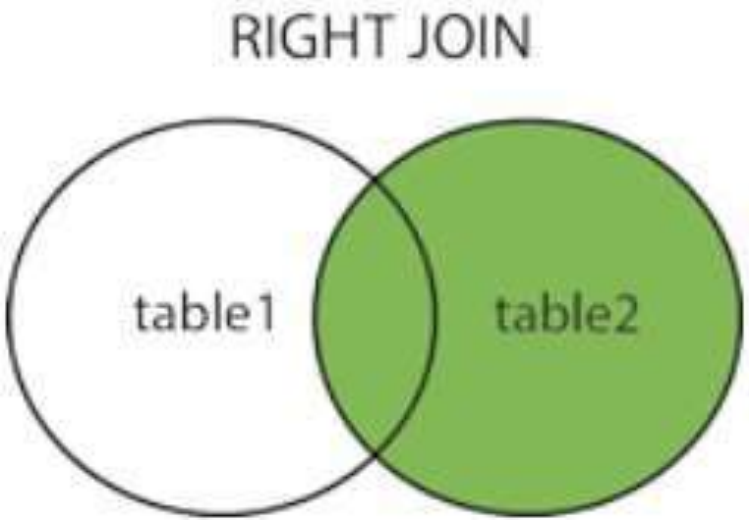
Sintaxis

SELECT columna_nombre(s)

FROM tabla1

RIGHT JOIN tabla2

ON tabla1.columna_nombre = tabla2.columna_nombre;



EJEMPLO RIGHT JOIN → TODOS LOS DE LA DERECHA, COINCIDENCIAS EN LA IZQUIERDA

```
SELECT Doctores.nombre, Citas.fecha, Citas.hora,
Pacientes.nombre AS paciente

FROM Doctores

RIGHT JOIN Citas ON Doctores.id = Citas.doctor_id

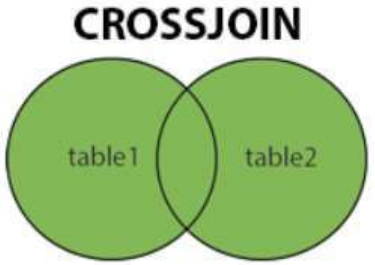
RIGHT JOIN Pacientes ON Citas.paciente_id =
Pacientes.id;
```

- Nombre del doctor (NULL si la cita no tiene un doctor asignado).
- Fecha de la cita (NULL si el paciente no tiene citas).
- Hora de la cita (NULL si el paciente no tiene citas).
- Nombre del paciente (Siempre aparecerá, ya que es el RIGHT JOIN más externo).

RIGHT JOIN es útil cuando quieres asegurar que todos los registros de la tabla derecha sean incluidos, incluso sin coincidencias en la izquierda.

CROSS JOIN

La palabra reservada ***CROSS JOIN*** devuelve todos los registros de ambas tablas (table1 y table2).



Sintaxis

SELECT columna_nombre(s)

FROM tabla1

CROSS JOIN tabla2;

La palabra reservada ***CROSS JOIN*** devuelve todos los registros coincidentes de ambas tablas, independientemente de que la otra tabla coincida o no. Por lo tanto, si hay filas en "Clientes" que no tienen coincidencias en "Pedidos", o si hay filas en "Pedidos" que no tienen coincidencias en "Clientes", esas filas también se enumerarán.

Si agrega una ***WHERE*** cláusula (si table1 y table2 tienen una relación), ***CROSS JOIN*** producirá el mismo resultado que la ***INNER JOIN*** cláusula.

ACTIVIDAD BÁSICA JOIN

- ¿Cómo obtener una lista de todas las citas que están confirmadas, mostrando el nombre del paciente, el nombre del doctor, la fecha y la hora de la cita, ordenadas por fecha ascendente?
- ¿Cómo obtener una lista de todos los pacientes que no tienen ninguna cita registrada en la base de datos?
- ¿Cómo listar las citas donde el paciente nació después del año 1990 y el estado de la cita sea "Pendiente" o "Confirmada"?
- ¿Cómo obtener una lista de todos los doctores cuya especialidad contenga la palabra "Neuro", mostrando también su número de licencia, teléfono y correo electrónico?

FUNCIONES SQL MIN () Y MAX ()

La MIN() función devuelve el valor más pequeño de la columna seleccionada.

La MAX() función devuelve el valor más grande de la columna seleccionada.

Sintaxis MIN()

SELECT MIN(column_name) FROM table_name WHERE condition;

Sintaxis MAX()

SELECT MAX(column_name) FROM table_name WHERE condition;

Ejemplo:

SELECT MIN(fecha) AS fecha_mas_antigua, MAX(fecha) AS fecha_mas_reciente

FROM Citas;

FUNCIONES SQL COUNT (), AVG () Y SUM ()

La COUNT() función devuelve el número de filas que coincide con un criterio específico.

Sintaxis:

```
SELECT COUNT(column_name)
FROM table_name
WHERE condition;
```

Ejemplo:

```
SELECT COUNT(*) AS total_citas
FROM Citas;
```

FUNCIÓN AVG()

La AVG() función devuelve el valor promedio de una columna numérica.

Sintaxis:

SELECT AVG (column_name) FROM table_name

WHERE condition;

Ejemplo:
SELECT AVG(precio) AS precio_promedio

FROM Servicios;

FUNCIÓN SUM()

La SUM() función devuelve la suma total de una columna numérica.

Sintaxis:

```
SELECT SUM(column_name) FROM table_name
WHERE condition;
```

Ejemplo:

```
SELECT SUM(Servicios.precio) AS ingresos_totales
FROM Citas
INNER JOIN Servicios ON Citas.servicio_id = Servicios.id;
```

LA SENTENCIA GROUP BY DE SQL

La GROUP BY declaración agrupa las filas que tienen los mismos valores en filas de resumen, como "encontrar el número de clientes en cada país".

La GROUP BY instrucción se utiliza a menudo con funciones de agregado (COUNT(), MAX(), MIN(), SUM(), AVG()) al agrupar el conjunto de resultados de una o más columnas.

SINTAXIS Y EJEMPLO GROUP BY

Sintaxis

```
SELECT column1, columnN
FROM table_name
WHERE condition
GROUP BY column;
```

Ejemplo:

```
SELECT estado, COUNT(*) AS total_citas
FROM Citas
GROUP BY estado;
```

ACTIVIDAD BÁSICA GROUP BY

- ¿Cuántas citas hay en cada estado (Confirmada, Pendiente, Cancelada)?
- ¿Cuántas citas ha atendido cada doctor?
- ¿Cuánto dinero ha generado cada servicio médico en total?

CLÁUSULA HAVING

La HAVING cláusula se agregó a SQL porque la WHERE palabra clave no se puede usar con funciones agregadas.

Sintaxis

```
SELECT column1, columnaN
FROM table_name
WHERE condition
GROUP BY column_name(s)
HAVING condition
```

EJEMPLO DE SQL HAVING

¿Cuáles son los servicios que han generado más de $200 en ingresos?

```
SELECT Servicios.nombre_servicio, SUM(Servicios.precio) AS ingresos_totales
FROM Citas
INNER JOIN Servicios ON Citas.servicio_id = Servicios.id
GROUP BY Servicios.nombre_servicio
HAVING ingresos_totales > 200;
```

ACTIVIDAD BÁSICA HAVING

- ¿Qué servicios tienen un precio promedio superior a $70?
- ¿Cuáles doctores han atendido más de 3 citas en total?
- ¿Cuáles servicios han generado más de $300 en ingresos totales? Ordenar los resultados de mayor a menor según ingresos.
- ¿Cuáles pacientes han tenido más de 2 citas confirmadas? Ordenar los resultados de mayor a menor según cantidad de citas.



TEMA 3: MANEJO DE DATOS CON PYTHON

Misión 1

EXPLICACIÓN DE CADA LIBRERÍA

- Pandas es una de las librerías que Python que permite realizar análisis y manipulación de datos, es muy potente, flexible y fácil de usa.
- NumPy al igual que pandas esta construida sobre Python, permite el manejo de matrices en n cantidad de dimensiones, además de facilitar el computo numero.





INTRODUCCIÓN A SERIES DE PANDAS

Una Series es una estructura de datos unidimensional de Pandas, similar a una lista o un arreglo de Numpy, pero con etiquetas (índices) para cada elemento.

Es ideal para representar una columna de datos, como una lista de edades, precios, nombres, etc.

```
import pandas as pd
serie = pd.Series([10, 20, 30, 40, 50])
print(serie)
```

CREACIÓN DE UN ‘DATAFRAME‘

Un DataFrame es una estructura bidimensional de Pandas que se parece a una tabla de Excel o una base de datos.

- Tiene filas y columnas
- Cada columna puede tener un tipo de dato diferente
- Es ideal para trabajar con conjuntos de datos reales

```python
import pandas as pd
datos = {
    "Producto" : ["Carro", "Moto", "Avion"],
    "Precio" : [300, 200, 700],
    "Cantidad" : [10, 20, 5]
}
df = pd.DataFrame(datos)
df
```

```python
# Ver las primeras filas (útil con datos grandes)
print(df.head())
# Ver las últimas filas
print(df.tail())
# Ver la forma del DataFrame (filas, columnas)
print("Dimensión:", df.shape)
# Ver tipos de datos por columna
print("Tipos de datos:\n", df.dtypes)
```

ACTIVIDAD PRACTICA DATAFRAME

- Crea un DataFrame con al menos 4 películas, incluyendo:
  - Título
  - Año de estreno
  - Calificación (de 1 a 10)
  - Muestra:
    - Los primeros 2 registros
    - El tipo de datos de cada columna

INTRODUCCIÓN A NUMPY + PANDAS

¿Qué es NaN?

NaN significa **"Not a Number"** y se utiliza para representar **valores faltantes o nulos** en un conjunto de datos. Pandas y Numpy lo reconocen automáticamente y lo manejan de forma eficiente.

```python
import pandas as pd
import numpy as np

#Crear datos con numpy (incluir NaN)
precios = np.array([100, 200, np.nan, 400, 500])
cantidades = np.array([10, np.nan, 30, np.nan, 50])

#Creación del diccionario
datos_nuevos = {
    "Producto" : ["Carro", "Moto", "Avion", "Barco", "Tren"],
    "Precio" : precios,
    "Cantidad" : cantidades
}

#Creación del DataFrame
df = pd.DataFrame(datos_nuevos)
print(df)
```

```python
# Ver qué valores están vacíos
print(df.isna())

# Contar valores faltantes por columna
print("Valores faltantes:\n", df.isna().sum())
```

TRATAMIENTO DE VALORES FALTANTES (NAN)

- Eliminar valores nulos: **`.dropna()`**
  - Esto elimina todas las filas que tengan al menos un valor faltante.
- Eliminar columnas con NaN
- Rellenar valores faltantes: **`.fillna()`**
  - Rellenar con la media, mediana, o moda

```
df_sin_nan = df.dropna()
```

```
df_sin_columnas_nan = df.dropna(axis=1)
```

```
# Reemplaza NaN con 0
df["Precio"] = df["Precio"].fillna(0)
# Media
df["Precio"] = df["Precio"].fillna(df["Precio"].mean())
# Mediana
df["Precio"] = df["Precio"].fillna(df["Precio"].median())
# Moda (el valor más frecuente)
df["Producto"] = df["Producto"].fillna(df["Producto"].mode()[0])
```

EXPLORACIÓN INICIAL DE DATOS

- Información general: **`.info()`**

```
#info() proporciona un resumen del DataFrame:
#Total de filas
#El total de columnas
#Tipo de datos de cada columna
#El uso de memoria
#Datos faltante
df.info()
```

- Estadísticas descriptivas: **`.describe()`**

```
#Funcion describe()
#Solo funciona con columnas numericas, y me da como resultado un resumen estadistico
df.describe()
```

```
print(df.describe(include='all'))
```

ACTIVIDAD PRACTICA DATAFRAME NAN

1. Crea un DataFrame con al menos 8 registros de productos (nombre, precio_unitario, cantidad, total).
2. Incluye algunos valores np.nan manualmente en todas las columnas.
3. Muestra:
   - ¿Cuántos valores faltantes hay por columna?
   - ¿Qué filas tienen datos incompletos?
   - El tipo de dato de cada columna
   - Valores estadísticos únicamente de las columnas numericas

ACCESO A ELEMENTOS EN UN DATAFRAME (.ILOC .LOC)

`.iloc[]` — Acceso por índice numérico

```
#Selección de la fila con posición [1]
print(df.iloc[[1, 3, 4]])
print("-----------------------")

#Seleción de valor especifico
#Seleccionando el 'Precio'[1] y la fila 3
print(df.iloc[3, 1])
print("-----------------------")

#Selección de un rango de filas y columnas
#Seleccionado los tres primeros filas y las dos primeras columnas
print(df.iloc[0:3, 0:2])
print("-----------------------")

#filtra todas las filas, y la condición es la columna
print(df.iloc[:, 2])
```

`.loc[]` — Acceso por etiqueta

```
#Seleccionando todas las columnas y una sola fila [1]
print(df.loc[1])
print("-----------------------")

#Seleción del registro en la posición [1], pero unicamente mostrando su columna 'Precio'
print(df.loc[1, 'Precio'])
print("-----------------------")

#Seleccionando un numero de filas especificos y unas columnas especificas 'Producto' 'Precio'
print(df.loc[1:3, ['Producto','Precio']])
print("-----------------------")

#Filtrar los datos con una condición, en la columna 'Precio' la condición es mayor a 150
print(df.loc[df['Precio'] > 150])
```

ACTIVIDAD PRACTICA POSICIONAMIENTO

1. Usa el DataFrame de productos.
2. Muestra:
    1. El nombre del segundo producto
    2. El precio del último producto
    3. Todas las columnas de la tercera fila
    4. La celda de fila 1, columna "Cantidad“
    5. ¿Cuáles productos cuestan más de $300?
    6. ¿Qué productos tienen más de 10 unidades disponibles?

CARGA DE ARCHIVO DE DATOS (.CSV)

```
import pandas as pd
#Leer y mostrar archivo .csv
df = pd.read_csv('/content/sampledata.csv')
df.head(10)
```

- Mostrar los últimos 10 registros del dataset
- Mostrar cuantas filas y columnas tiene este dataset
- Identificar cuantos datos faltantes hay por columna
- Mostrar información detallada de todo el dataset (columnas, filas, faltantes, tipo de datos, cantidad de memoria usada)
- Mostrar un resumen estadístico únicamente de las columnas numéricas del dataset
- Selecciona un rango de filas y columnas con (iloc)

CARGA DE ARCHIVO DE DATOS (SQL)

```python
import sqlite3
import pandas as pd
#Función para ejecutar un archivo SQL
def execute_sql_script(conn, filepatch):
  with open(filepatch, 'r', encoding='utf-8') as file:
    sql_script = file.read()
  try:
    conn.executescript(sql_script)
    print('El script se ejecuto de manera correcta')
  except sqlite3.OperationalError as e:
    print(f'Error al ejecutar el script: {e}')

#Conección a la base de datos
conn = sqlite3.connect('/content/SectorMineroEnergeticoColombia.db')
#Ejecutar el archivo SQL
execute_sql_script(conn, '/content/SectorMineroEnergeticoColombia.sql')

#Consulta SQL para obtener los datos de una tabla
query = "SELECT * FROM eficiencia_energetica;"
#Crear el DataFrame con base a la consulta
df_eficiencia_energetica = pd.read_sql_query(query, conn)
#Mostrar las primeras 5 filas
df_eficiencia_energetica.head(5)
```

- Importación de librerías
- Función personalizada: **`execute_sql_script`**
  - **`conn`** conexión a la base de datos
  - **`filepatch`** ruta del archivo .sql que contiene las instrucciones
- Lectura del archivo SQL
  - Se abre el archivo **`.sql`** en modo lectura **`('r')`** con codificación **`'utf-8'`**.
  - Se guarda todo el contenido del archivo en la variable **`sql_script`**.
- Ejecución del script SQL y manejo de errores

ACTIVIDAD DE CARGUE DE DATOS SQL

1. Investiga qué tablas hay en la base de datos

```
cursor = conn.cursor()
#Consulta el nombre de las tablas de la base de datos
cursor.execute("SELECT name FROM sqlite_master WHERE type = 'table';")
tablas = cursor.fetchall()
#Imprimir el nombre de las tablas
for table in tablas:
  print(table[0])
```

2. Crea una sentencia SQL para extraer la mayor cantidad registros y columnas (hacer uso de join)
3. Explora la estructura y el contenido del DataFrame
   a) `.info()`
   b) `.describe()`
   c) `.isna`
4. Si aplica rellenar valores faltantes según el tipo de dato.

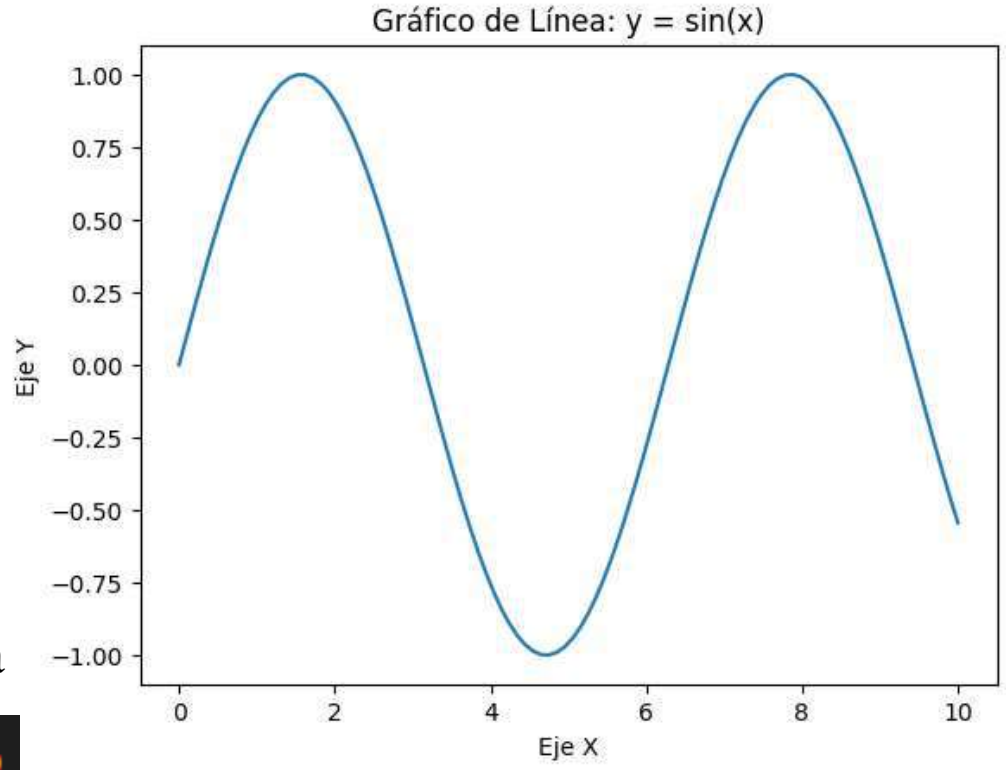
GRÁFICO DE LÍNEA — PLT.PLOT

Un gráfico de línea se usa para mostrar tendencias o cambios continuos a lo largo del tiempo o un rango.

```python
import matplotlib.pyplot as plt
import numpy as np

x = np.linspace(0, 10, 100)  # 100 valores entre 0 y 10
y = np.sin(x)                # función seno

plt.plot(x, y)
plt.xlabel("Eje X")
plt.ylabel("Eje Y")
plt.title("Gráfico de Línea: y = sin(x)")
plt.show()
```



Usa **`color`**, **`linestyle`**, y **`marker`** para personalizar la línea

```python
plt.plot(x, y, color="red", linestyle="--", marker="o")
```

GRÁFICO DE BARRAS — PLT.BAR

Se utiliza para comparar cantidades entre categorías.

```python
import matplotlib.pyplot as plt
import numpy as np

categorias = ['A', 'B', 'C', 'D']
valores = [10, 25, 15, 30]

plt.bar(categorias, valores, color='skyblue')
plt.xlabel("Categorías")
plt.ylabel("Valores")
plt.title("Gráfico de Barras")
plt.show()
```

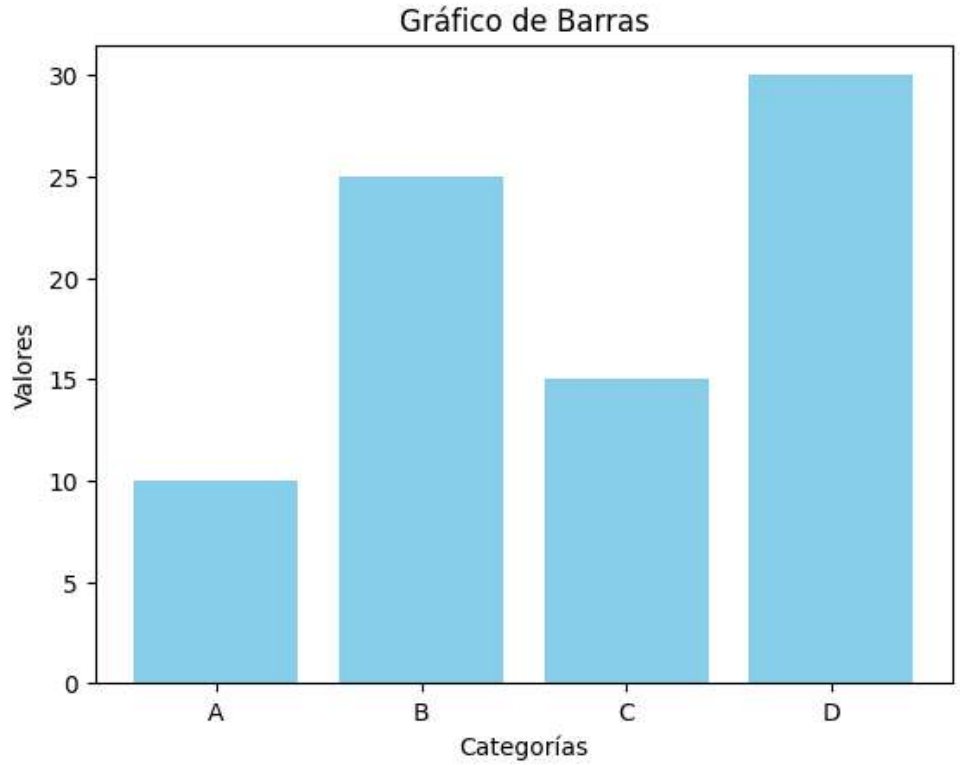


GRÁFICO DE DISPERSIÓN — PLT.SCATTER Y SNS.SCATTERPLOT

Muestra la relación entre dos variables numéricas.

```python
import matplotlib.pyplot as plt
import numpy as np

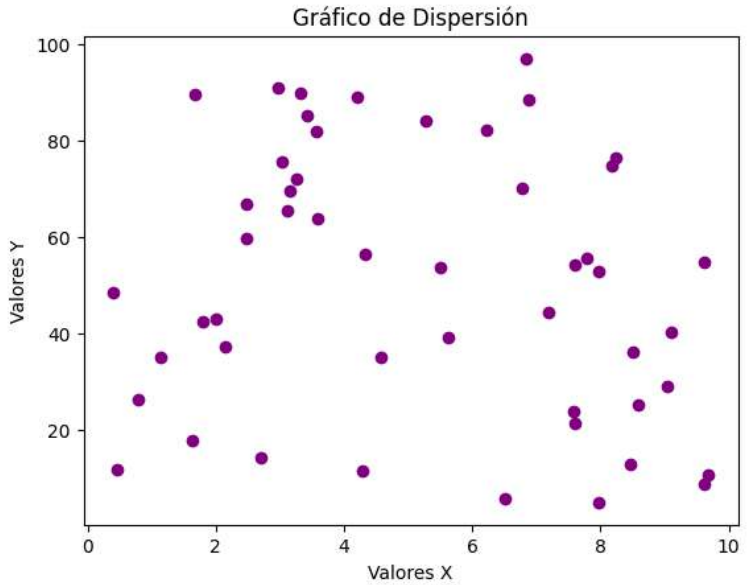
x = np.random.rand(50) * 10
y = np.random.rand(50) * 100

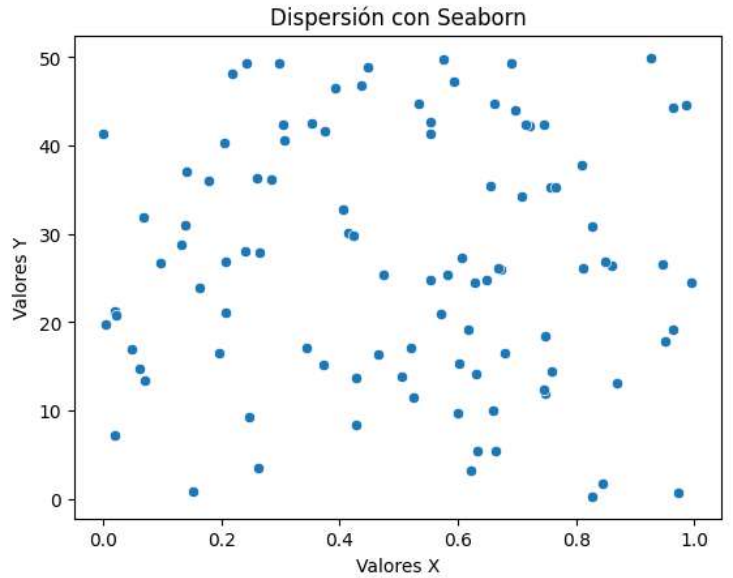
plt.scatter(x, y, color='purple')
plt.xlabel("Valores X")
plt.ylabel("Valores Y")
plt.title("Gráfico de Dispersión")
plt.show()
```

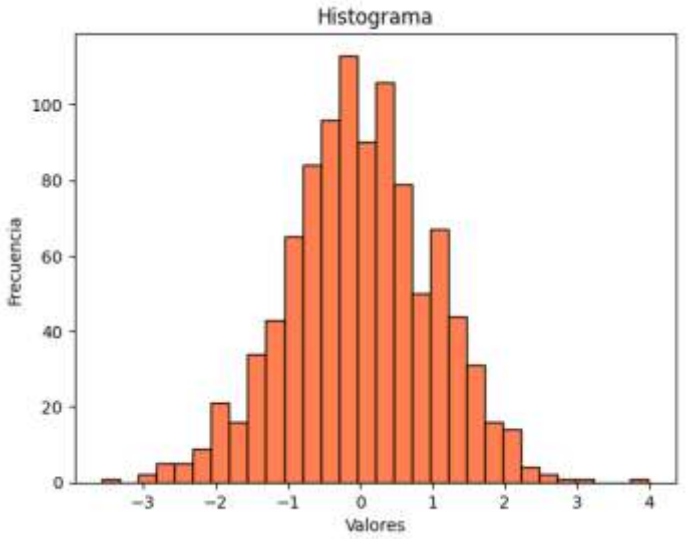
```python
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

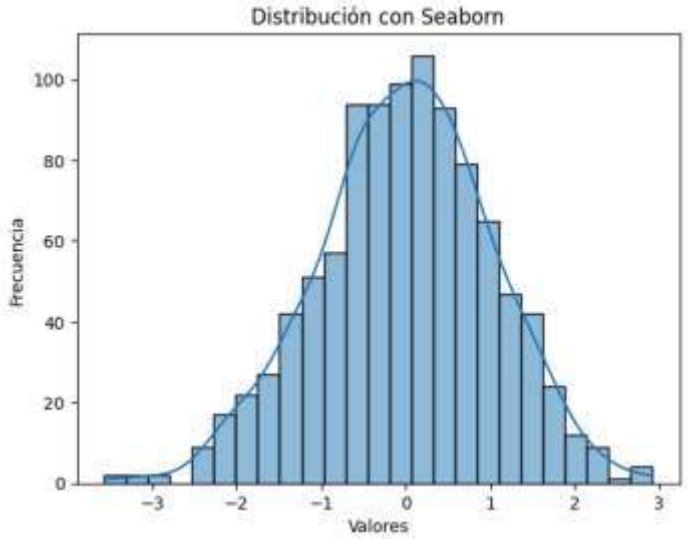
datos = pd.DataFrame({
    "X": np.random.rand(100),
    "Y": np.random.rand(100) * 50
})

sns.scatterplot(x="X", y="Y", data=datos)
plt.xlabel("Valores X")
plt.ylabel("Valores Y")
plt.title("Dispersión con Seaborn")
plt.show()
```









HISTOGRAMA — PLT.HIST Y SNS.HISTPLOT

Muestra la distribución de una variable (frecuencia)

```python
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

datos = np.random.randn(1000)

plt.hist(datos, bins=30, color='coral', edgecolor='black')
plt.xlabel("Valores")
plt.ylabel("Frecuencia")
plt.title("Histograma")
plt.show()
```

```python
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

datos = np.random.randn(1000)

sns.histplot(datos, kde=True)
plt.xlabel("Valores")
plt.ylabel("Frecuencia")
plt.title("Distribución con Seaborn")
plt.show()
```

MAPA DE CALOR (CORRELACIÓN) — SNS.HEATMAP

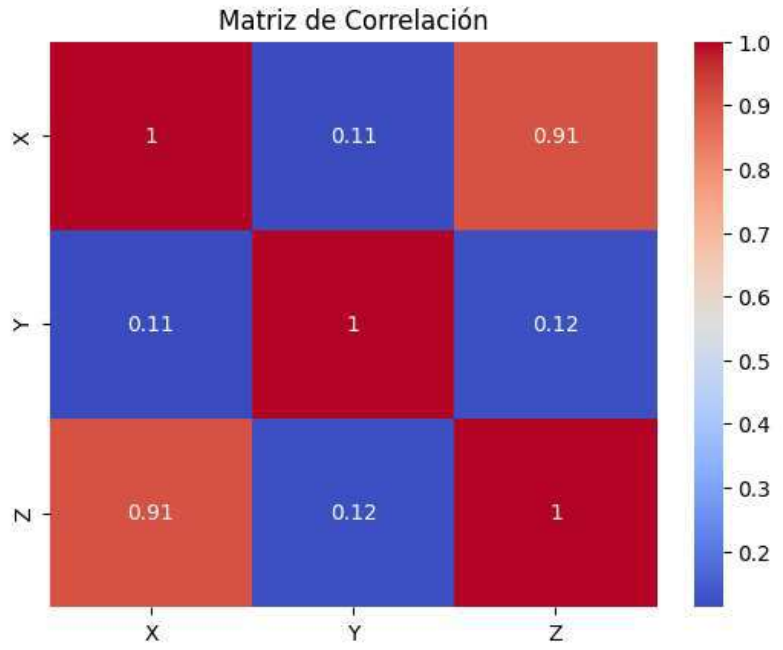
Muestra la **relación estadística (correlación)** entre variables numéricas.

```python
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

np.random.seed(0)
df = pd.DataFrame({
    "X": np.random.randn(100),
    "Y": np.random.randn(100) * 50
})
df["Z"] = df["X"] * 10 + np.random.randn(100) * 5

# Matriz de correlación
correlacion = df.corr()

# Mapa de calor
sns.heatmap(correlacion, annot=True, cmap="coolwarm")
plt.title("Matriz de Correlación")
plt.show()
```



ACTIVIDAD VISUALIZACIÓN CON SAMPLEDATA.CSV

1. Explora brevemente el conjunto de datos con **`.info()`**, **`.describe()`**, para entender qué variables puedes graficar.
2. Elige dos tipos de gráficos diferentes de los que vimos en clase o investiga otros disponibles en la documentación de Matplotlib o Seaborn.
3. Crea ambos gráficos usando columnas del DataFrame.
   a) Un gráfico de barras comparando categorías.
   b) Un histograma de una variable numérica.
   c) Un gráfico de dispersión entre dos variables.
   d) Un gráfico de caja (boxplot), línea, pastel, etc.
4. Al final, escribe una celda de Markdown respondiendo:
   a) ¿Qué variables graficaste?
   b) ¿Qué tipo de gráfico usaste y por qué?
   c) ¿Qué observas en el gráfico? ¿Hay patrones, valores extremos, agrupaciones, etc.?